Pioneer Delivery — Part II

Task D: Validation and Algorithm Analysis

Task A — Closest delivery locations

I sort the points by x and y once. Each recursive call splits the x-sorted points in half and divides the y-sorted list by point ID, so tied x-values are handled correctly. The two halves are solved recursively. The combine step checks points in a narrow strip around the dividing line, in y-order, against at most the next seven points. This geometric bound avoids comparing every strip pair. Small groups (up to three points) use brute force. Time: O(n log n); auxiliary space: O(n) for sorted lists, partitions, and recursion.

Task B — Dispatch-order inversions

I first replace each ID in the actual order with its position in the planned order. Merge sort recursively counts inversions in each half. During merging, when a right-side value is smaller than the next left-side value, it forms an inversion with every remaining left-side value. Time: O(n log n); auxiliary space: O(n) for merge arrays and recursive slices. Python integers also support the required 4,999,950,000 count.

Task C — kth smallest delivery time

I divide the values into groups of at most five and sort only those small groups. I recursively select the median of their medians as a deterministic pivot, then partition values into less-than, equal-to, and greater-than groups. If k is in the lower group, I recurse there. If it is within the equal group, I return the pivot. Otherwise I recurse into the upper group using k - len(lower) - len(equal). The pivot guarantees a constant fraction of values can be discarded, giving O(n) worst-case time. This list-based implementation uses O(n) auxiliary space.

Correctness testing

I tested all required cases: A1–A4 (squared distances 25, 1, 16, 9); B1–B4 (counts 2, 0, 6, 0), plus the 100,000-item reversed case; and C1–C5 (values 16, 4, 35, 7, 3). Additional cases include tied closest pairs, a single inversion, sorted and reverse-sorted input, and many repeated times. For small random inputs, I compared Task A with brute-force pair distances, Task B with nested-loop inversion counting, and Task C with sorted(values)[k-1]. All tests passed.

Performance measurements

Approximate execution time in milliseconds, measured locally with Python 3 using `benchmark.py` (one run per size).

n	Task A (ms)	Task B (ms)	Task C (ms)
100	0.279	0.111	0.057
1,000	3.024	1.561	0.395
5,000	74.612	10.874	1.123

Tasks A and B generally grow faster than linearly, consistent with O(n log n); Task A has a noticeable jump at 5,000 points in this run. Task C grows more slowly, consistent with linear-time selection. Actual timings vary by machine, input, and run.